

Üüritung — Estonian Rental Market Monitoring Platform

Team Members: Hashim | Sudais | Calvin | Daboikiabo

University of Tartu — Spring 2026

Part 1 — Realistic System

What it does: Üüritung is an Estonian rental market monitoring platform that aggregates apartment listings from major Estonian portals (KV.ee, City24, Rendin), tracks rental price trends by neighborhood and city, and helps tenants find affordable housing through personalized alerts, neighborhood comparisons, and landlord reputation data. The platform serves the Estonian rental market, where finding transparent, up-to-date information about fair rental prices is a significant challenge, especially for students and newcomers.

Who uses it:

- **Tenants** (students, expats, families) searching for rentals with price alerts, neighborhood insights, and landlord reviews.
- **Landlords** who list properties and want to understand market pricing to set competitive rates.
- **Real estate analysts** and researchers tracking price trends, vacancy rates, and market dynamics across Estonian cities.
- **Platform administrators** managing scraping schedules, user reports, and data quality.

Main Functionalities:

- **Listing Aggregation:** scrape and normalize rental listings from KV.ee, City24, and Rendin into a unified database.
- **Price Analytics:** compute average prices, price-per-square-meter trends, and vacancy rates per neighborhood and city over time.
- **User Alerts:** let tenants register personalized alerts (by neighborhood, max price, min size) and receive email or in-app notifications when matching listings appear.
- **Neighborhood Profiles:** provide detailed neighborhood information including average rent, transport links, amenities score, and resident reviews.
- **Landlord Reputation:** aggregate tenant reviews and complaint history per landlord to promote transparency.
- **User Accounts:** manage registration, authentication, saved searches, and alert preferences.
- **Listing Management:** allow landlords to claim and manage their own listings with verified status.

- **Notifications:** deliver email and in-app alerts for new listings, price drops, review responses, and system announcements.

Why Multiple Microservices Are Necessary:

Üüriturg spans fundamentally different business domains: web scraping and data ingestion, statistical analytics, user identity management, real-time alerting, neighborhood data curation, landlord reputation tracking, listing ownership, and cross-cutting notification delivery. Each domain has distinct data models, update frequencies, and scaling requirements. The scraping service must run on scheduled intervals and handle external site changes without affecting user-facing features. Analytics requires batch computation over large datasets, while alerts demand near-real-time event processing. Landlord reviews and neighborhood profiles evolve independently from listing data. Combining these into a monolith would create tight coupling between components with incompatible lifecycles — a scraping failure should not bring down user authentication, and redeploying analytics logic should not require restarting the alerting pipeline. Microservice separation ensures independent scaling, fault isolation, and clear data ownership, while integration between services coordinates cross-domain workflows such as: scraping triggers analytics recalculation, new listings trigger alert matching, and landlord verification requires both user identity and listing ownership data.

Part 2 — Architecture

Service Overview

#	Service	Owner	Responsibility	Key Relationships
1	Scraper Service	Hashim	Scrape KV.ee, City24, Rendin; normalize & store listings	Publishes events to Analytics and Alert services
2	Analytics Service	Hashim	Compute price trends, averages, vacancy rates per neighborhood	Reads from Scraper; serves data to frontend
3	User Service	Sudais	Registration, authentication, profile & saved searches	Used by all services for identity verification
4	Alert Service	Sudais	Manage user alert preferences; match new listings to alerts	Consumes Scraper events; calls User & Notification
5	Neighborhood Service	Calvin	Neighborhood profiles, amenity scores, transport data	Reads from Analytics; serves neighborhood pages
6	Landlord Service	Calvin	Landlord profiles, tenant reviews, reputation scores	Calls User Service; linked to Listing Service
7	Listing Service	Daboikiabo	Landlord-managed listings, claim verification, listing status	Depends on Scraper, User, and Landlord services
8	Notification Service	Daboikiabo	Email & in-app message delivery, templating, delivery tracking	Called by Alert, Landlord, and Listing services

Service Responsibilities & Dependencies

1. Scraper Service (Hashim)

What it does: Crawls Estonian rental portals (KV.ee, City24, Rendin) on a configurable schedule, extracts listing data (price, size, rooms, address, neighborhood), normalizes it into a unified schema, deduplicates across sources, and stores raw listings. When new or updated listings are detected, it publishes `listing.new` and `listing.updated` events to the message broker.

Data Managed: Listing records (listing ID, source, external ID, title, price, size, price/m², neighborhood, address, rooms, URL, scrape timestamp, active status). Scrape job metadata (last run, status, error count).

Why it is separated: Web scraping is inherently fragile and depends on external site structure. Isolating it ensures that changes in portal HTML or scraping failures do not affect user-facing services. It also allows independent scaling of scraping frequency without impacting the rest of the system.

Dependencies (What it Calls): None — the Scraper Service is a data producer with no upstream dependencies.

Dependencies (What Calls It): Analytics Service (reads listing data via REST), Alert Service (consumes `listing.new` events), Listing Service (reads scraped data for landlord claim matching).

2. Analytics Service (Hashim)

What it does: Reads listing data from the Scraper Service via REST, computes daily and weekly aggregates (average price, average price/m², listing count, vacancy trends) per neighborhood and city, stores time-series snapshots, and exposes endpoints for charts and summary statistics.

Data Managed: Analytics snapshots (neighborhood, date, average price, average price/m², listing count, median price, price change percentage). City-level summary statistics.

Why it is separated: Analytics computation is batch-oriented and CPU-intensive. Separating it from the scraping pipeline ensures that heavy aggregation queries do not slow down data ingestion or user-facing reads.

Dependencies (What it Calls): Scraper Service (reads listing data for aggregation).

Dependencies (What Calls It): Neighborhood Service (reads analytics data for neighborhood profiles), Frontend (reads charts and summaries).

3. User Service (Sudais)

What it does: Manages user registration, login, profile management, and saved search preferences. Acts as the central identity provider — all other services call it to verify user identity and retrieve contact information.

Data Managed: User records (user ID, name, email, phone, role [tenant/landlord/admin], registration date, preferences, saved searches).

Why it is separated: User identity is a cross-cutting concern referenced by every other service. Centralizing it prevents data duplication and ensures a single source of truth for authentication and authorization.

Dependencies (What it Calls): None — the User Service is the central identity hub with no upstream dependencies.

Dependencies (What Calls It): Alert Service (retrieves user contact info and preferences), Landlord Service (verifies landlord identity), Listing Service (verifies user roles), Notification Service (retrieves recipient contact details).

4. Alert Service (Sudais)

What it does: Allows tenants to register alert rules (target neighborhood, maximum price, minimum size, room count). Consumes listing.new events from the message broker, evaluates each new listing against all active alert rules, and triggers the Notification Service when a match is found.

Data Managed: Alert rules (alert ID, user ID, neighborhood filter, max price, min size, min rooms, creation date, active status). Match history (alert ID, listing ID, matched timestamp).

Why it is separated: Alert matching is event-driven and must process new listings in near-real-time. Separating it from the Scraper ensures that matching logic can evolve independently (e.g., adding ML-based relevance scoring) without modifying the data ingestion pipeline.

Dependencies (What it Calls): User Service (retrieves user preferences and contact info), Notification Service (triggers email/in-app alerts).

Dependencies (What Calls It): Scraper Service (publishes listing.new events consumed by Alert Service).

5. Neighborhood Service (Calvin)

What it does: Maintains detailed profiles for each tracked neighborhood: description, average rent, amenity scores (grocery stores, public transport, parks, schools), safety ratings, and resident reviews. Provides comparison endpoints for users evaluating where to live.

Data Managed: Neighborhood records (neighborhood ID, city, name, description, latitude/longitude, amenity scores, transport links). Resident reviews (review ID, user ID, neighborhood ID, rating, text, date).

Why it is separated: Neighborhood data comes from multiple sources (analytics aggregates, external APIs, user reviews) and has a different update cadence than listings. Separating it allows curated neighborhood content to be managed independently from real-time listing data.

Dependencies (What it Calls): Analytics Service (reads price statistics for the neighborhood), User Service (verifies reviewer identity).

Dependencies (What Calls It): Frontend (serves neighborhood comparison pages).

6. Landlord Service (Calvin)

What it does: Manages landlord profiles with aggregated reputation data: tenant reviews, complaint history, response rate, average listing quality score. Landlords can register, claim their listings, and respond to reviews.

Data Managed: Landlord profiles (landlord ID, user ID, name, contact, verification status, average rating). Tenant reviews (review ID, landlord ID, reviewer user ID, rating, text, date). Complaint records.

Why it is separated: Reputation tracking involves sensitive review data with its own moderation rules and lifecycle. Keeping it separate ensures that review disputes or moderation changes do not affect listing or analytics services.

Dependencies (What it Calls): User Service (verifies landlord and reviewer identity).

Dependencies (What Calls It): Listing Service (retrieves landlord reputation for listing display), Notification Service (sends alerts on new reviews to landlords).

7. Listing Service (Daboikiabo)

What it does: Handles landlord-managed listings: landlords can claim scraped listings as their own, post new direct listings, update listing details, and mark listings as rented. Manages listing verification and enrichment (adding photos, floor plans, verified status).

Data Managed: Managed listings (listing ID, landlord ID, scraped listing reference, verified status, photos, description, availability dates). Claim requests (claim ID, landlord ID, listing ID, status).

Why it is separated: Landlord-managed listing operations (claiming, editing, verifying) have different authorization rules and workflows than automated scraping. Separating them avoids mixing automated data ingestion with user-driven content management.

Dependencies (What it Calls): Scraper Service (reads raw listing data for claim matching), User Service (verifies landlord authorization), Landlord Service (retrieves landlord profile for display).

Dependencies (What Calls It): Notification Service (sends claim approval notifications to landlords).

8. Notification Service (Daboikiabo)

What it does: Delivers email and in-app notifications to users. Receives notification requests from other services, formats messages using templates, tracks delivery status, and manages notification preferences (opt-in/opt-out, frequency, channels).

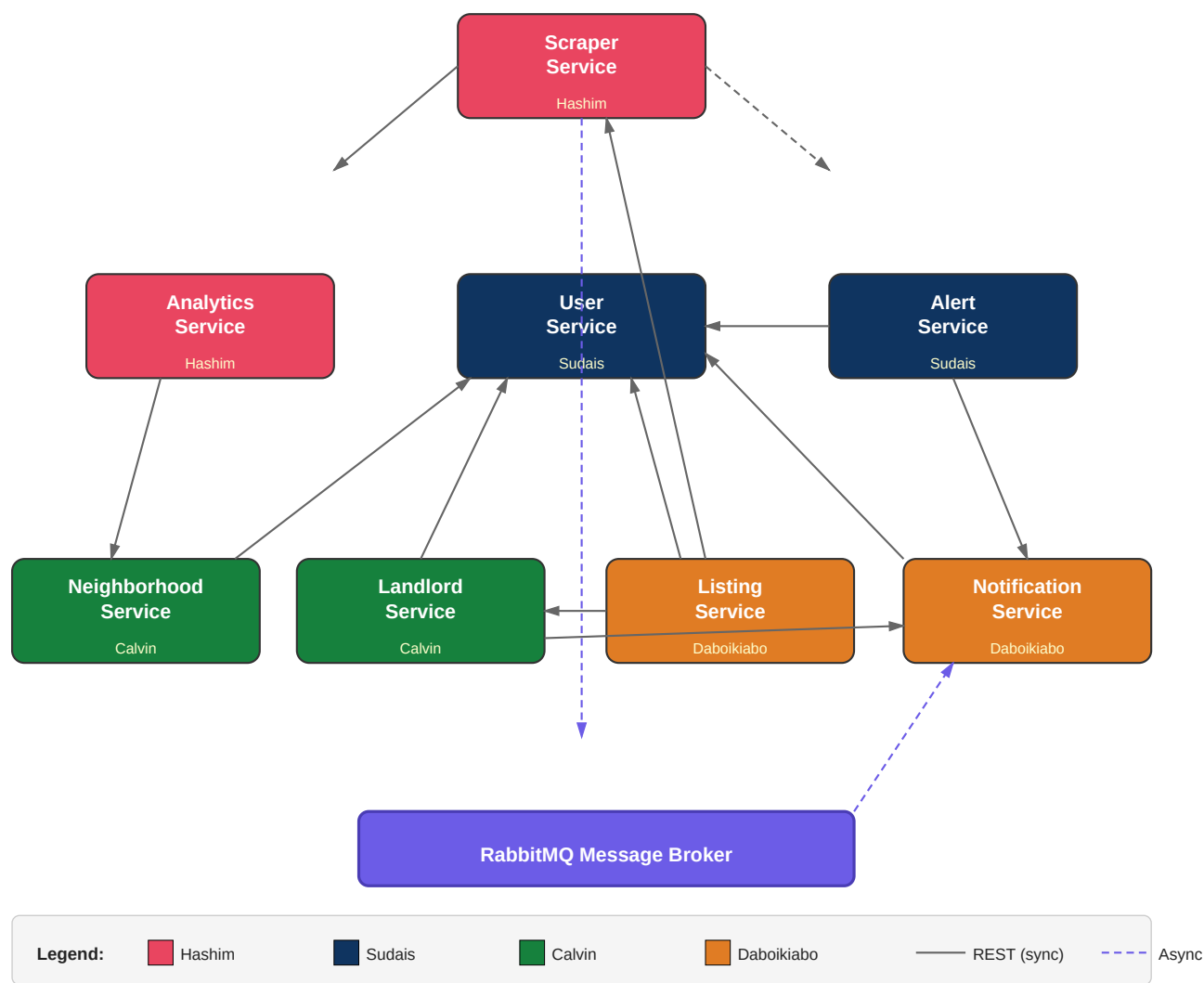
Data Managed: Notification records (notification ID, recipient user ID, channel [email/in-app], subject, body, timestamp, delivery status). Message templates. User notification preferences.

Why it is separated: Notification delivery is a cross-cutting infrastructure concern used by multiple services. Centralizing it avoids duplicating email/messaging logic across Alert, Landlord, and Listing services, and ensures consistent formatting and delivery tracking.

Dependencies (What it Calls): User Service (retrieves recipient contact information for message delivery).

Dependencies (What Calls It): Alert Service (triggers new listing match alerts), Landlord Service (triggers review notification), Listing Service (triggers claim status notifications).

Architecture Diagram



Dependency Summary

Service	Calls (depends on)	Called by
Scraper Service	—	Analytics, Alert, Listing
Analytics Service	Scraper	Neighborhood, Frontend
User Service	—	Alert, Landlord, Listing, Notification, Neighborhood
Alert Service	User, Notification	Scraper (via broker)
Neighborhood Service	Analytics, User	Frontend
Landlord Service	User	Listing, Notification
Listing Service	Scraper, User, Landlord	Notification
Notification Service	User	Alert, Landlord, Listing